MODEL CONTEXT PROTOCOL • FASTMCP & AGENTIC TOOLS

Model Context Protocol (MCP) in Agentic Systems

Building FastMCP Servers, Orchestrating Dual Transports (stdio & streamable-http), and Executing MultiServerMCPClient ReAct Agents

An authoritative, publication-grade technical study guide deconstructing the Model Context Protocol (MCP). Explores the standardized protocol bridging AI models and enterprise tools, FastMCP server development, local standard input/output console pipes (`stdio`), distributed Server-Sent Events (`streamable-http`), unified tool aggregation via `MultiServerMCPClient` , and Windows Jupyter IPC optimization.

Target Notebook: `3-MCP.ipynb`

Generation Date: October 31, 2026

Language & Runtime: Python 3.14 • Virtualenv (.venv)

Core Frameworks: MCP 1.30+ • FastMCP • LangChain 1.4.7 • LangGraph • Groq LPU

B. Learning Objectives

After studying this notebook and study guide, you will master:

- **The Model Context Protocol (MCP) Paradigm:** Understand why MCP is becoming the open universal standard for connecting LLM agents to external tools, databases, and context repositories.
- **Rapid Server Development with FastMCP:** Build lightweight, type-safe MCP servers using Python decorators (`@mcp.tool()`) that automatically generate JSON-RPC tool schemas.
- **Subprocess Pipe Transport (`stdio`):** Configure secure, zero-network inter-process communication using operating system standard input/output streams.
- **Distributed Web Transport (`streamable-http`):** Deploy ASGI-backed MCP web microservices using Starlette, Uvicorn, and Server-Sent Events (SSE) for remote or containerized environments.
- **Unified Client Orchestration:** Aggregate disparate MCP servers across different transport layers into a single LangChain tool list via `MultiServerMCPClient` .
- **Autonomous ReAct Agent Execution:** Bind dynamically fetched MCP tools into a LangGraph ReAct agent to solve multi-step mathematical problems and external queries.
- **Windows Subprocess Debugging:** Diagnose and resolve the Windows Jupyter `io.UnsupportedOperation: fileno` issue when managing background stdio processes.

C. Prerequisites, Environment & Security Setup

> **Runtime & API Configuration**
>
> This notebook operates on **Python 3.14**. It communicates with local and network MCP servers using the official Anthropic `mcp` Python SDK, connects to **Groq LPU** for sub-second inference, and leverages `langchain-mcp-adapters` for LangChain integration.

Component	Version / Identifier	Source / Purpose
`mcp`	`>=1.30.0`	Official Model Context Protocol SDK, including `FastMCP` server framework
`langchain-mcp-adapters`	`>=0.3.2`	Adapter translating MCP JSON-RPC tool definitions into LangChain `BaseTool` objects
`langgraph`	`>=0.3.0`	Agent graph execution engine, `create_react_agent`
`langchain-groq`	`>=1.1.3`	High-throughput LPU model inference (Groq chat completions)
`uvicorn / starlette`	Installed via `mcp`	Asynchronous ASGI web server powering `streamable-http` SSE endpoints

> **Security & API Key Masking Protocol**
>
> Ensure `GROQ_API_KEY` is loaded securely from `.env` . Never hardcode credentials in server scripts or notebook cells.

D. Comprehensive Topic Overview

Before the introduction of the **Model Context Protocol (MCP)**, connecting an AI agent to multiple tools required writing custom bespoke integration code for every provider. If you had 5 tools and 3 different agent frameworks, you needed up to 15 different adapters. MCP introduces an open standard (analogous to the Language Server Protocol in software engineering) that decouples tool development from agent execution.

1. The Three Layers of MCP Architecture

- **The Host Application:** The client software (e.g., LangGraph agent, Claude Desktop, or custom Python application) that coordinates the user session and prompts the LLM.
- **The Client Adapter:** The component (such as `MultiServerMCPClient`) that manages transport connections, translates JSON-RPC 2.0 messages, and exposes native tool abstractions.
- **The MCP Servers:** Lightweight, independent programs that expose tools, resources (data sources), and prompts via standardized JSON-RPC protocols.

Figure 1: Model Context Protocol (MCP) Multi-Server Architectural Overview

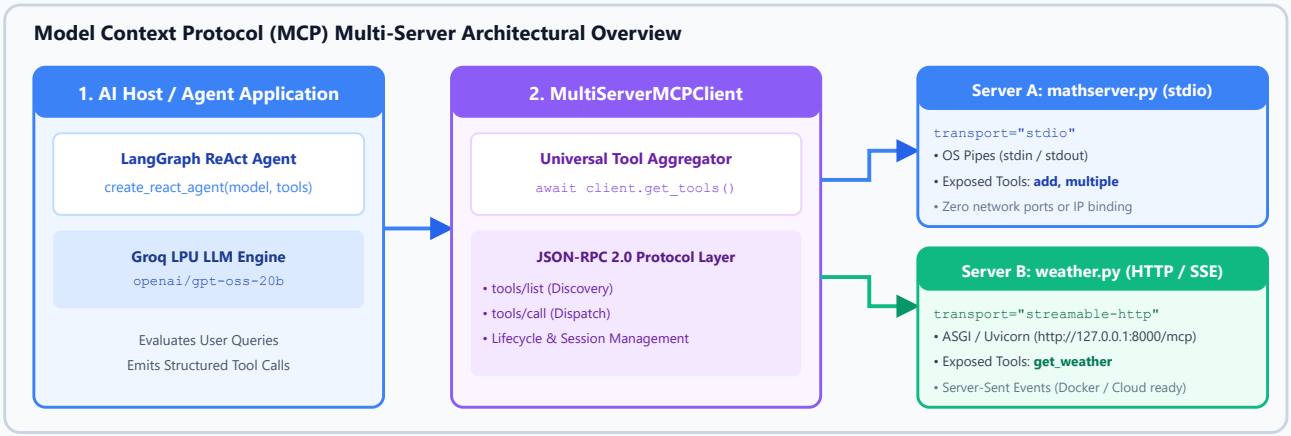


Figure 1 Architectural Breakdown: *Visual topology showing how a single LangGraph ReAct agent aggregates tools from both a local* `stdio` *child process (Server A) and a remote* `streamable-http` *web service (Server B).*

2. Comparing Transports: `stdio` vs `streamable-http`

MCP supports two primary communication transports, each tailored to distinct operational and security requirements:

- `stdio` **(Standard Input/Output):** The client executes the server script as a child process using the operating system's process management APIs. Communication occurs over anonymous memory pipes. It requires zero network ports, has zero external attack surface, and provides maximum performance for local execution.
- `streamable-http` **(HTTP + Server-Sent Events):** The server runs as an independent ASGI web service listening on a network port (e.g., `http://127.0.0.1:8000/mcp`). Communication uses HTTP POST for commands and SSE for asynchronous streaming responses. This enables multi-client sharing, containerization (Docker/Kubernetes), and cloud deployment.

E. Detailed Notebook Walkthrough

The source notebook `3-MCP.ipynb` guides you through the creation of two separate MCP servers, starting a background service, orchestrating them via `MultiServerMCPClient`, and executing an autonomous ReAct agent.

Phase 1: Server 1: `mathserver.py` via stdio (Cells 0–2)

Cell 2 • Defining and Writing mathserver.py

```
%%writefile mathserver.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Math")

@mcp.tool()
def add(a: int, b: int) -> int:
    \"\"\"Add two numbers.\"\"\"
    return a + b

@mcp.tool()
def multiple(a: int, b: int) -> int:
    \"\"\"Multiply two numbers.\"\"\"
    return a * b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Detailed Code Analysis:

- `%%writefile mathserver.py`: The IPython cell magic writes this code to disk as a standalone Python file. This is crucial because a `stdio` MCP server must run in its own subprocess with clean standard input/output streams, rather than inside the interactive notebook kernel.
- `mcp = FastMCP("Math")`: Instantiates the FastMCP application with the identifier `"Math"`.
- `@mcp.tool()`: The decorator inspects function signatures, parameter type annotations (`a: int, b: int`), and docstrings (`\"\"\"Add two numbers.\"\"\"`) to automatically generate an OpenAI-compatible JSON schema.
- `mcp.run(transport="stdio")`: Launches the server listener. When executed via CLI or subprocess, it reads incoming JSON-RPC requests from `sys.stdin` and writes responses to `sys.stdout`.

Phase 2: Server 2: `weather.py` via streamable-http (Cells 3–4)

Cell 4 • Defining and Writing weather.py

```
%%writefile weather.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Weather")

@mcp.tool()
async def get_weather(location: str) -> str:
    \"\"\"Get the weather location.\"\"\"
    return f"It's always raining in {location}"

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

Detailed Code Analysis:

- `transport="streamable-http"`: Instead of blocking on standard input, FastMCP starts a high-performance Uvicorn web server listening on `127.0.0.1:8000` with the endpoint `/mcp`.
- `async def get_weather(...)`: FastMCP natively supports asynchronous Python functions, making it ideal for non-blocking I/O operations such as external weather APIs or database queries.

Phase 3: Launching the Background Web Server (Cells 5–6)

Cell 6 • Starting Uvicorn in a Background Subprocess

```
import subprocess
import time
import sys

if 'weather_process' in locals():
    weather_process.terminate()

weather_process = subprocess.Popen([sys.executable, "weather.py"])
print("Started weather.py web server. Waiting 3 seconds for Uvicorn to bind")

time.sleep(3)

print("Weather server ready")
```

Operational Mechanics:

- `sys.executable`: Ensures that the background process uses the exact same Python virtual environment (`.venv`) where `mcp` and `uvicorn` are installed.
- `time.sleep(3)`: Gives the Uvicorn ASGI server sufficient time to bind port 8000 before the client initiates network handshake requests.
- **Safety Guard:** The `if 'weather_process' in locals()` block terminates any previously running instance, preventing `[Errno 10048] address already in use` socket errors when re-executing cells.

Phase 4: Client Orchestration & ReAct Agent (Cells 7–8)

Cell 8 • MultiServerMCPClient & ReAct Invocations

```
import os
import sys
import asyncio
from dotenv import load_dotenv
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent
from langchain.chat_models import init_chat_model

# Fix Windows Jupyter fileno() issue: restore underlying OS stderr for subprocess creation
if hasattr(sys, "__stderr__") and sys.__stderr__ is not None:
    sys.stderr = sys.__stderr__

load_dotenv()
os.environ["GROQ_API_KEY"] = os.getenv("GROQ_API_KEY")

model = init_chat_model("groq:openai/gpt-oss-20b")

async def run_agent():
    # Initialize MultiServerMCPClient connecting to both servers
    client = MultiServerMCPClient({
        "math": {
            "command": sys.executable,  # Uses the current Python runtime
            "args": ["mathserver.py"],
            "transport": "stdio",
        },
        "weather": {
            "url": "http://localhost:8000/mcp", # HTTP endpoint
            "transport": "streamable_http",
        }
    })

    # Fetch all tools dynamically from both servers
    tools = await client.get_tools()
    print(f"Loaded {len(tools)} tools: {[t.name for t in tools]}")

    # Construct the ReAct agent graph with the model and tools
    agent = create_react_agent(model, tools)

    # Query 1: Arithmetic Decomposition
    print("\nQuery 1: Math")
    math_response = await agent.ainvoke({
        "messages": [{"role": "user", "content": "what's (3 + 5) x 12?"}]
    })
    print("Math response:", math_response["messages"][-1].content)

    # Query 2: Remote HTTP Tool Query
    print("\nQuery 2: Weather")
    weather_response = await agent.ainvoke({
        "messages": [{"role": "user", "content": "what is the weather in California?"}]
    })
    print("Weather response:", weather_response["messages"][-1].content)

# Execute the async agent function
await run_agent()
```

Execution Trace & Output:

```
Loaded 3 tools: ['add', 'multiple', 'get_weather']

Query 1: Math
Math response: (3 + 5) * 12 = 8 * 12 = 96.

Query 2: Weather
Weather response: The weather report indicates that it is currently raining in California.
```

How the Agent Multi-Hop Execution Worked:

1. `client.get_tools()` dynamically queried both the `stdio` subprocess and the `http://localhost:8000/mcp` endpoint. It retrieved tool schemas for `add`, `multiple`, and `get_weather`.
2. In Query 1 (`(3 + 5) x 12`), the Groq model recognized that it needed arithmetic decomposition. It called `add(a=3, b=5)` over the `stdio` pipe, received `8`, and then called `multiple(a=8, b=12)` to obtain `96`.
3. In Query 2 (`weather in California`), the model recognized the geographical intent and dispatched an HTTP POST request to `weather.py`, calling `get_weather(location="California")`.

Phase 5: Cleanup & Process Termination (Cells 9–10)

Cell 10 • Clean Process Termination

```
if 'weather_process' in locals():
    weather_process.terminate()
    print("Terminated weather server background process.")
```

Lifecycle Management: Terminating the background Uvicorn server frees port 8000, preventing orphaned background processes from consuming CPU cycles or interfering with subsequent notebook runs.

F. End-to-End Execution Flow & Architectural Synthesis

Figure 2: Stdio vs Streamable-HTTP Transports Comparison

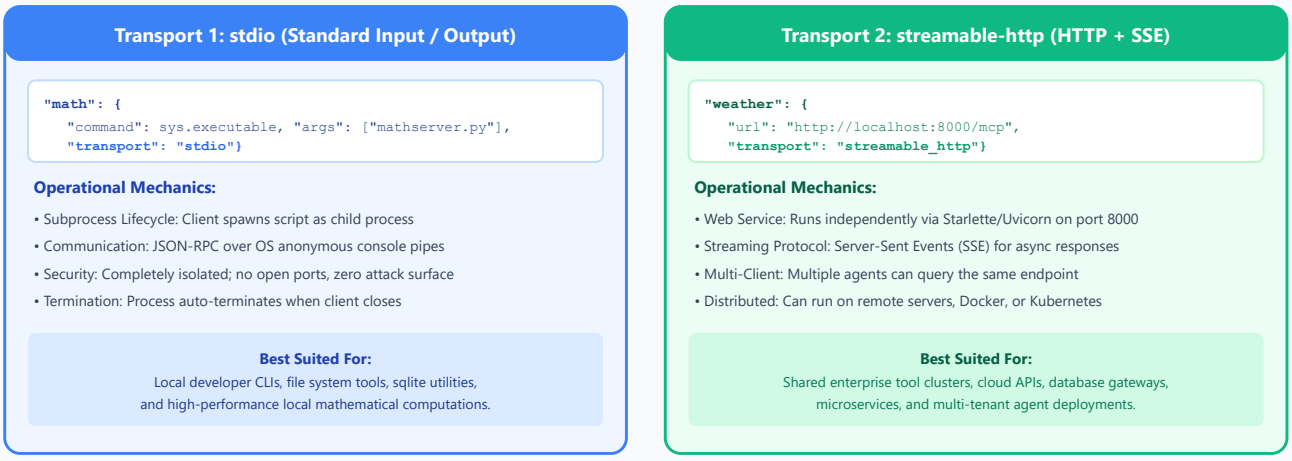


Figure 2 Architectural Breakdown: *Comparative matrix analyzing operational mechanics, security isolation, and optimal enterprise deployment scenarios for local OS stdio pipes versus distributed streamable-http microservices.*

G. Reference Guide: Important APIs & Classes

Class / Method	Package	Key Parameters	Return / Behavior
`FastMCP(name)`	`mcp.server.fastmcp`	`name: str`	Initializes an MCP server instance with automated tool registration and schema generation.
`@mcp.tool()`	`mcp.server.fastmcp`	None	Function decorator that extracts type hints and docstrings into an MCP JSON-RPC tool schema.
`mcp.run(transport=...)`	`mcp.server.fastmcp`	`transport: "stdio" | "streamable-http" | "sse"`	Starts the MCP server using the specified transport layer (blocking synchronous loop).
`MultiServerMCPClient(connections)`	`langchain_mcp_adapters.client`	`connections: dict`	Unified client managing multiple heterogeneous MCP server connections simultaneously.
`client.get_tools()`	`langchain_mcp_adapters.client`	None	Asynchronously queries all registered MCP servers and returns a unified list of LangChain `BaseTool` objects.

`create_react_agent(model, tools)`	`langgraph.prebuilt`	`model: BaseChatModel, tools: list`	Constructs an autonomous LangGraph state machine with ReAct reasoning and dynamic tool calling.

H. Common Errors, Debugging & Troubleshooting

Pitfall 1: Windows Jupyter fileno() UnsupportedOperation

Symptom: `io.UnsupportedOperation: fileno` when calling `client.get_tools()`.
Root Cause: In Jupyter on Windows, `ipykernel` replaces `sys.stderr` with an in-memory `OutStream` that lacks an OS file descriptor. When MCP's `create_windows_process` attempts to query `sys.stderr.fileno()` to spawn the child process, it crashes.
Fix:

```
if hasattr(sys, "__stderr__") and sys.__stderr__ is not None:
    sys.stderr = sys.__stderr__
```

Figure 3: Diagnosing the Windows Jupyter fileno() Architecture

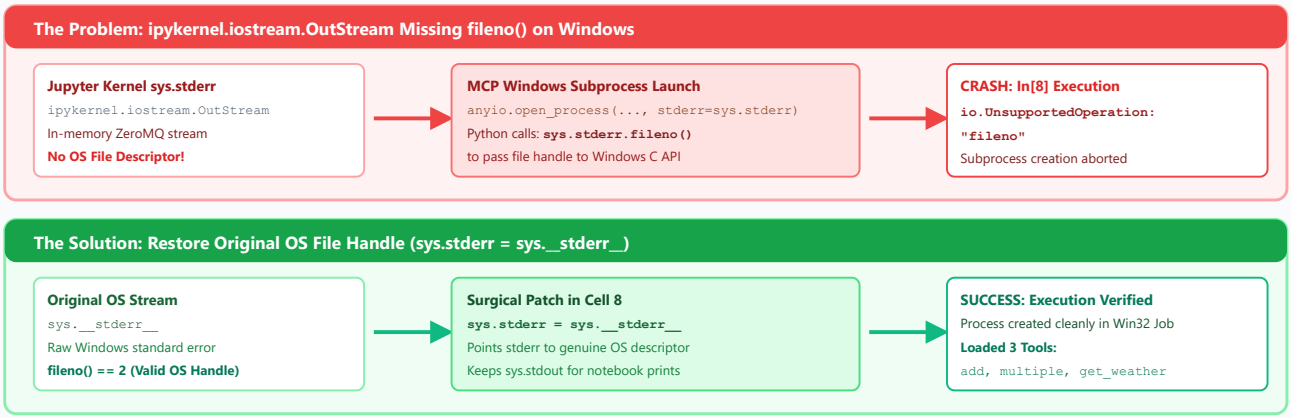


Figure 3 Architectural Breakdown: *How `ipykernel`'s OutStream causes subprocess creation failure on Windows, and how restoring `sys.__stderr__` establishes a valid OS file descriptor.*

Pitfall 2: Running mcp.run() Directly in Jupyter Without %%writefile

Symptom: `RuntimeError: Already running asyncio in this thread`.
Root Cause: `mcp.run(transport="stdio")` internally invokes `anyio.run()`, which cannot be nested inside Jupyter's already-active asyncio event loop.
Fix: Use `%%writefile server.py` at the top of the cell and run the script as an external subprocess.

Pitfall 3: Port 8000 Socket Collisions

Symptom: `[Errno 10048] error while attempting to bind on address ('127.0.0.1', 8000)`.
Root Cause: Re-running Cell 6 without terminating the previous `weather.py` process holding port 8000.
Fix: Add `if 'weather_process' in locals(): weather_process.terminate()` before calling `Popen`.

I. Practical Extensions & Industry Best Practices

- **Tool Namespace Isolation:** When connecting to dozens of MCP servers, tool name collisions can occur (e.g. multiple servers exposing a `search` tool). Initialize the client with `MultiServerMCPClient(..., tool_name_prefix=True)` to automatically prefix tools with their server name (e.g., `math_add`, `weather_get_weather`).
- **Dynamic Authentication & Headers:** For cloud-hosted MCP servers, supply authorization tokens or custom headers in the connection specification:

```
"weather": {
    "url": "https://api.enterprise.com/mcp",
    "transport": "streamable_http",
    "headers": {"Authorization": "Bearer <token>"}
}
```

- **Containerized MCP Deployment:** Package FastMCP servers into lightweight Docker containers exposing port 8000. Agents running in Kubernetes can dynamically connect to tool services via internal DNS.

J. Key Takeaways & Conceptual Map

Core Principles of Model Context Protocol (MCP)

1. **Universal Tool Standard:** MCP decouples tool creation from agent orchestration. Write a tool once in FastMCP; connect it to any agent framework.
2. **Dual Transports for Dual Needs:** Use `stdio` for local, isolated, ultra-secure developer tools; use `streamable-http` for scalable, distributed enterprise microservices.
3. **Single Aggregator:** `MultiServerMCPClient` seamlessly handles subprocess spawning, network sessions, and tool translation.
4. **Autonomous Execution:** LangGraph's `create_react_agent` automatically determines which tools to call across multiple connected servers to fulfill complex user prompts.

K. Practice Exercises & Knowledge Checks

Question 1 (Conceptual): Why does `stdio` transport require no network ports, and what makes it inherently more secure for local developer workstations than HTTP?

Question 2 (Code Tracing): If a user asks the agent: *"Add 15 and 25, then check the weather in Tokyo"*, describe the sequence of tool invocations and transports used by `MultiServerMCPClient`.

Question 3 (Debugging): A developer runs a FastMCP server with `streamable-http` in a Docker container, but the client cannot connect to `http://localhost:8000/mcp`. What are the two most likely configuration mistakes?

Question 4 (Implementation Challenge): Write a FastMCP server script `system_info.py` with `transport="stdio"` that exposes a tool `get_os_info()` returning the current Python version and platform name.

Answers & Complete Explanations

Answer 1: `stdio` relies on operating system anonymous pipes created directly between the parent process and child process. Because it binds no IP address or TCP port, it cannot be probed, intercepted, or attacked over the local network or internet.

Answer 2:

1. The agent invokes `add(a=15, b=25)` via the `stdio` transport, receiving `40`.
2. The agent invokes `get_weather(location="Tokyo")` via the `streamable-http` transport over port 8000.
3. The agent synthesizes both outputs into the final response.

Answer 3: 1. The container failed to publish port 8000 (missing `-p 8000:8000`).
2. The FastMCP server inside the container was bound to `127.0.0.1` instead of `0.0.0.0`, preventing connections from outside the container.

Answer 4:

```
from mcp.server.fastmcp import FastMCP
import platform, sys

mcp = FastMCP("SystemInfo")

@mcp.tool()
def get_os_info() -> str:
    \"\"\"Return platform and Python version details.\"\"\"
    return f"Platform: {platform.platform()}, Python: {sys.version.split()[0]}"

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

L. Further Learning & Official Documentation

- Model Context Protocol Official Website: Specifications, SDKs, and architecture.
- Anthropic MCP Python SDK GitHub: FastMCP server documentation and code examples.
- LangChain MCP Adapters Documentation: Official guide on integrating MCP clients into LangChain and LangGraph.